

Python Course

Simple Summary of All 59 PDFs

Written for absolute beginners.
No heavy words, no tough English.
Every topic comes with a small example you can try yourself.

What is inside	59 PDFs explained one by one
Total pages in your merged file	342
You will learn	Python, SQL, Data Analysis, Machine Learning, Deep Learning, Web Basics (CSS) and Gen AI
Who can read this	Anyone. Even if you have never written a line of code.

Before You Start

Read this page first. It takes 3 minutes.

How to use these three files

You now have three things:

- **Python_Course_All_59_PDFs_Merged.pdf** - your full textbook. All 59 PDFs are joined one after another in the same order (PDF 1, PDF 2, ... PDF 59). Nothing inside them was changed.
- **This summary file** - a simple, friendly explanation of every PDF. Read a topic here first, then open that topic in the big file.
- **The extracted folder** - all 59 original PDFs kept separately, exactly as they were.

Every topic below ends with a line like *'In the big merged file, this starts on page 12'*. Use that number to jump straight to the original notes.

What is programming, in one minute?

A computer is very fast but not very smart. It only does what you tell it, step by step. That step-by-step instruction list is called a **program**, and writing it is called **programming**.

Think of it like giving a recipe to a cook who has never cooked before. You cannot say 'make tea'. You must say: boil water, add tea leaves, add milk, add sugar, strain, serve. Programming is exactly this - breaking a job into tiny, clear steps.

What is Python?

Python is a language we use to talk to the computer. It looks very close to English, so beginners learn it faster than most other languages. Python is used for websites, data analysis, automation, machine learning, artificial intelligence and much more.

```
print("Hello, world!")

# This single line tells the computer:
# "show this text on the screen"
```

That is a real Python program. One line, and it works.

Three words you must know

- **Library (or package)**: Ready-made code written by other people so you do not have to write everything yourself. Example: Pandas helps you read Excel-like data. You just install it and use it.
- **Terminal (or Command Prompt)**: A black window where you type commands instead of clicking.
- **pip**: The tool that installs libraries. You write `pip install pandas` in the terminal and Pandas gets downloaded to your computer.

How to set up Python on your computer

```
# 1. Download and install Python from python.org
#    (tick the box "Add Python to PATH" while installing)

# 2. Check it works. Type this in the terminal:
python --version

# 3. Install a library you need. Example:
pip install numpy pandas matplotlib

# 4. Run your first program (save it as hello.py):
python hello.py
```

Most students also install **VS Code** (to write code) or use **Jupyter Notebook** (to write code in small blocks and see results immediately). Either is fine.

The five building blocks of every program

- **Data** - the information you work with (a name, a number, a list of marks).
- **Variables** - named boxes that hold that data.
- **Decisions** - 'if this is true, do that'.
- **Repetition** - 'do this 100 times' (loops).
- **Functions** - a named bundle of steps you can reuse again and again.

Almost everything in this course is a combination of these five ideas. Keep them in mind and the whole course becomes much easier.

Part 1 - Python Fundamentals

PDFs 1 to 10 - the foundation of everything

PDF 1 Python Fundamentals Part 1

What is this? Your very first lesson. It covers four things: how to show something on screen (output), how to take something from the user (input), how to store data in variables, and the types of data Python understands. A variable is simply a named box: you put a value inside it and use the name later. Data types tell Python whether your value is a whole number (int), a decimal (float), text (str), a true/false value (bool), or empty (None). The notes also explain operators - the symbols used for maths and comparisons.

Example:

```
name = input("What is your name? ") # take input
age = int(input("Your age: "))      # convert text to number

print("Hello", name)                # show output
print("Next year you will be", age + 1)

print(type(age))                    # <class 'int'>
print(17 // 5)                      # 3   (whole part of division)
print(17 % 5)                       # 2   (remainder)
print(2 ** 3)                       # 8   (2 to the power 3)
```

input() always gives you text. That is why we wrap it in int() when we need a number.

Why it matters: Everything after this - loops, functions, files, Pandas, machine learning - is built on these four ideas. If this PDF is clear, half the course becomes easy.

In the big merged file, this starts on page 3.

PDF 2 Python Fundamentals - Assignment 1

What is this? Ten practice questions on the basics. They look small but they teach the habits that matter: taking input, converting types, doing maths, and explaining why the answer came out the way it did. One question asks you to explain $10 + 3 * 2 ** 2$ - this tests operator precedence: power first, then multiply, then add, so the answer is 22, not 52.

Example:

```
# Q: Celsius to Fahrenheit
c = float(input("Temperature in Celsius: "))
f = (c * 9/5) + 32
print("Fahrenheit:", f)

# Q: swap two numbers without a third variable
a, b = 5, 9
a, b = b, a      # Python swaps in one line
print(a, b)      # 9 5
```

Why it matters: Do not skip assignments. Reading code makes sense; writing code makes you a programmer.

In the big merged file, this starts on page 10.

PDF 3 Python Fundamentals Part 2

What is this? Now your program starts to think. **Conditions** let it choose: if the user is 18 or older, allow voting, otherwise refuse. **Loops** let it repeat: print numbers 1 to 100 without writing 100 lines. **Functions** let you bundle

steps under a name and reuse them. The notes also show logical operators (and, or, not), nesting (an if inside an if), the one-line ternary form, and match-case, which is a cleaner way to write a long if-elif chain.

Example:

```
def greet(name, times=1):          # a reusable function
    for i in range(times):         # loop
        print("Hello", name)

greet("Aman", 3)

# conditions
age = int(input("Age: "))
if age >= 18:
    print("You can vote")
elif age >= 13:
    print("Almost there")
else:
    print("Too young")

# one-line if (ternary)
status = "Adult" if age >= 18 else "Child"
```

Why it matters: Loops plus conditions plus functions is the moment you stop writing scripts and start writing programs.

In the big merged file, this starts on page 12.

PDF 4 Python Fundamentals - Assignment 2

What is this? Ten questions on conditions, loops and functions. You will calculate tax slabs, print even numbers in a range, pull digits out of a number, build a small calculator, check whether a number is prime, and build a number-guessing game. These are classic beginner problems because they force you to think in steps.

Example:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

print(is_prime(7))  # True
print(is_prime(9))  # False
```

The digit trick: the last digit of a number is `n % 10`, and you remove it with `n // 10`.

Why it matters: The calculator and guessing game are the first programs that feel like real software.

In the big merged file, this starts on page 23.

PDF 5 Python Fundamentals Part 3

What is this? Strings, Lists, Tuples, Dictionaries and Sets - the containers that hold collections of data. A **string** is text and cannot be changed after creation. A **list** is an ordered, changeable collection. A **tuple** is like a list but locked - once made, it cannot be changed. A **dictionary** stores pairs, like a real dictionary stores a word and its meaning. A **set** keeps only unique values and has no order. The notes also cover indexing, slicing, and modern string formatting with f-strings.

Example:

```
s = "Python"
print(s[0])      # P      (indexing starts at 0)
print(s[2:])     # thon   (slicing)
print(s[::-1])   # nohtyP (reversed)

marks = [88, 92, 75]      # list - can change
marks.append(99)

point = (10, 20)          # tuple - cannot change

student = {"name": "Riya", "age": 21}  # dictionary
print(student["name"])    # Riya

unique = {1, 2, 2, 3}     # set
print(unique)             # {1, 2, 3}

print(f"{student['name']} is {student['age']} years old")
```

Why it matters: Real data almost never comes as a single value. These five containers are how you hold it.

In the big merged file, this starts on page 25.

PDF 6 Python Fundamentals - Assignment 3

What is this? Ten questions that make you use the containers. You will check whether a word is a palindrome, average a list of numbers, merge and sort two lists, split a tuple into even and odd numbers, build a menu-driven student record system with a dictionary, and use sets to find duplicates and unique characters.

Example:

```
word = input("Enter a word: ")
if word == word[::-1]:
    print("Palindrome!")
else:
    print("Not a palindrome")

# find numbers that appear more than once
nums = [1, 2, 3, 2, 4, 3, 3]
seen, dup = set(), set()
for n in nums:
    if n in seen:
        dup.add(n)
    seen.add(n)
print(dup)      # {2, 3}
```

Why it matters: The menu-based dictionary program is your first mini application - the same pattern runs ATMs and kiosks.

In the big merged file, this starts on page 35.

PDF 7 Python Fundamentals Part 4 - Object Oriented Programming

What is this? A big step up in thinking. Instead of scattering data across many lists, you create your own type. A **class** is a blueprint; an **object** is one real thing made from that blueprint. The class 'Student' is the design; 'Riya with roll number 21' is an object. Inside a class, variables are called **attributes** and functions are called **methods**. The notes also cover the four pillars: encapsulation (protect data), inheritance (reuse a parent class), polymorphism (same method name, different behaviour) and abstraction (show what, hide how).

Example:

```

class Student:
    def __init__(self, name, marks):    # constructor
        self.name = name
        self.marks = marks

    def grade(self):                    # method
        return "A" if self.marks >= 90 else "B"

s1 = Student("Riya", 95)
s2 = Student("Aman", 78)
print(s1.name, s1.grade())    # Riya A
print(s2.name, s2.grade())    # Aman B

class CollegeStudent(Student):        # inheritance
    def __init__(self, name, marks, branch):
        super().__init__(name, marks)
        self.branch = branch

```

Why it matters: Every serious Python project - web apps, ML pipelines, games - is organised with classes.

In the big merged file, this starts on page 38.

PDF 8 Python Fundamentals - Assignment 4

What is this? Ten OOP practice problems: a BankAccount with deposit and withdraw, a Book with reviews, a Student with validation in getters and setters, a Shape family where Circle, Rectangle and Triangle each override area(), a Vehicle parent with Car and Bike children, an abstract Employee class, a Person class with default parameters, a Player class that counts how many players exist, multiple inheritance, and finally a small chat system built from User, Message and ChatRoom classes.

Example:

```

class Shape:                        # parent
    def area(self):
        pass

class Circle(Shape):                # child overrides
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r

class Rectangle(Shape):
    def __init__(self, l, b):
        self.l, self.b = l, b
    def area(self):
        return self.l * self.b

for s in [Circle(7), Rectangle(4, 5)]:
    print(s.area())                # same call, different result

```

Why it matters: The chat system question is a genuine mini project - finish it and you can show it off.

In the big merged file, this starts on page 49.

PDF 9 Python Fundamentals Part 5

What is this? Four very practical topics. **File I/O** is how your program remembers data after it closes - you open a file in a mode (r to read, w to write, a to append), read or write, then close it. **Exception handling** stops your program from crashing when something unexpected happens, like a missing file or division by zero. **List comprehensions** are a short, clean way to build a new list from an old one. **The JSON module** lets you save

and load Python dictionaries, which is how apps store settings and exchange data on the internet.

Example:

```
import json

# writing a file (with = automatically closes it)
with open("notes.txt", "w") as f:
    f.write("Hello students!")

# reading it back safely
try:
    with open("data.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("File not found!")
finally:
    print("Done checking.")

# list comprehension: keep numbers above 15
nums = [5, 10, 15, 20, 25]
big = [n for n in nums if n > 15]      # [20, 25]

# save and load JSON
json.dump({"city": "Delhi"}, open("c.json", "w"))
print(json.load(open("c.json")))
```

Why it matters: Saving data and handling errors is what separates a classroom script from a real program.

In the big merged file, this starts on page 52.

PDF 10 Python Fundamentals - Assignment 5

What is this? Five practice tasks on files, errors and JSON: write five names into a file and read them back, keep adding log lines to a file, filter a list with a comprehension, save a dictionary of cities to a JSON file and update it, and catch a `FileNotFoundError` gracefully.

Example:

```
import json

cities = {"Delhi": 32000000, "Pune": 7400000}
json.dump(cities, open("cities.json", "w"))

cities = json.load(open("cities.json"))
cities["Jaipur"] = 4100000      # add a new one
json.dump(cities, open("cities.json", "w"))
print(cities)
```

Why it matters: This is exactly how real apps store user settings and saved data.

In the big merged file, this starts on page 59.

Part 2 - Working With Data (NumPy and Pandas)

PDFs 11 to 13 - the tools every data job expects you to know

PDF 11 NumPy

What is this? NumPy stands for Numerical Python. It gives you a super-charged list called an **array**. Arrays are much faster than normal lists because they store all values in one continuous block of memory and are written in C behind the scenes. The biggest idea is **vectorization**: instead of looping over every item yourself, you write one operation and NumPy applies it to the whole array at once.

Example:

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a * 2)          # [ 2  4  6  8 10] - no loop needed
print(a + 10)         # [11 12 13 14 15]

m = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array (matrix)
print(m.shape)        # (2, 3)
print(m[0, 1])        # 2

print(np.zeros((3, 4))) # 3x4 filled with 0
print(np.arange(0, 10, 2))# [0 2 4 6 8]
print(a.mean(), a.sum(), a.max())
```

On ten million numbers, NumPy is roughly 20 to 50 times faster than a Python loop.

Why it matters: Pandas, Matplotlib, Scikit-learn, TensorFlow - almost every data library is built on top of NumPy arrays.

In the big merged file, this starts on page 60.

PDF 12 Pandas Notes

What is this? Pandas is the tool you use for table-shaped data, like an Excel sheet inside Python. It gives you two structures. A **Series** is one column. A **DataFrame** is the whole table with rows, columns and labels. With Pandas you can read a CSV file in one line, look at the first rows, get statistics, filter rows, group data, handle missing values, and join two tables.

Example:

```
import pandas as pd

df = pd.read_csv("students.csv") # read a file
print(df.head())                # first 5 rows
print(df.shape)                 # (rows, columns)
print(df.info())                # column types + missing values
print(df.describe())            # count, mean, min, max...

print(df[df["Marks"] > 80])      # filter rows
print(df.groupby("City")["Marks"].mean())# group and average
df["Grade"] = df["Marks"].apply(lambda m: "A" if m > 90 else "B")
df = df.dropna()                # drop missing values
```

Why it matters: If you work with data in Python, you will spend most of your time inside a DataFrame.

In the big merged file, this starts on page 72.

What is this? Two hands-on practice sets on famous datasets. With the **Iris** flower data you display rows, filter rows, group by species and create a new calculated column. With the **Titanic** data you select columns, filter female passengers with expensive tickets, and compute survival rates by passenger class.

Example:

```
import pandas as pd
df = pd.read_csv("titanic.csv")

print(df[["Name", "Sex", "Age", "Fare", "Survived"]].head())

rich_women = df[(df.Sex == "female") & (df.Fare > 30)]
print(len(rich_women))

print(df.groupby("Pclass").agg(
    survival_rate=("Survived", "mean"),
    avg_fare=("Fare", "mean")))
```

Why it matters: Filtering and grouping are the two operations data analysts use all day long.

In the big merged file, this starts on page 88.

Part 3 - SQL (Databases)

PDFs 14 to 17 - how real companies store and fetch data

PDF 14 SQL Notes

What is this? SQL is the language you use to talk to a database. A database is an organised store of data, and a relational database keeps it in tables that are linked to each other by keys. With SQL you create tables, add rows, read rows, update them and delete them - the famous CRUD. The notes cover data types, keys and constraints, and the main query clauses: SELECT to choose columns, WHERE to filter rows, ORDER BY to sort, GROUP BY to summarise, HAVING to filter the summary, LIMIT to cap results, and LIKE, IN and BETWEEN for pattern and range matching.

Example:

```
SELECT name, city
FROM students
WHERE marks > 80
ORDER BY marks DESC
LIMIT 10;

-- average marks for each city, only cities with 5+ students
SELECT city, AVG(marks) AS avg_marks, COUNT(*) AS total
FROM students
GROUP BY city
HAVING COUNT(*) >= 5;

-- names starting with A
SELECT * FROM students WHERE name LIKE 'A%';
```

SQL keywords are not case sensitive, and a semicolon ends a statement.

Why it matters: Almost every company keeps its data in a database. If you can write SQL, you can get that data.

In the big merged file, this starts on page 90.

PDF 15 SQL Cheat Sheet

What is this? A one-page reference of every command, grouped into families: DDL to define structure (CREATE, ALTER, DROP, TRUNCATE), DML to change rows (INSERT, UPDATE, DELETE), DQL to query (SELECT with WHERE, ORDER BY, GROUP BY, HAVING), and the clauses for joins, unions, subqueries and indexes. Keep it open while you practise - nobody memorises all of this.

Example:

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
);

INSERT INTO students VALUES (1, 'Aman', 92);
UPDATE students SET marks = 95 WHERE id = 1;
DELETE FROM students WHERE id = 1;
SELECT * FROM students;
```

Why it matters: Even senior developers keep a cheat sheet handy. This is normal, not cheating.

In the big merged file, this starts on page 116.

PDF 16 SQL Assignment

What is this? Ten queries to run on an Employee table with columns for id, name, department, salary and hire date. You will display all employees, select only three columns, filter by department and by salary, sort by hire date, list unique departments, match names starting with 'A', use BETWEEN, calculate an average, and count employees per department while keeping only departments with more than three people.

Example:

```
SELECT department, COUNT(*) AS staff
FROM employees
GROUP BY department
HAVING COUNT(*) > 3;

SELECT AVG(salary) FROM employees;
SELECT DISTINCT department FROM employees;
```

Why it matters: HAVING after GROUP BY is the part most beginners get wrong - practise it here.

In the big merged file, this starts on page 121.

PDF 17 SQL 2 Notes

What is this? The advanced half of SQL. **Transactions** let you run several statements as one unit, controlled by the ACID rules - all of them succeed or none do. You start with START TRANSACTION, finish with COMMIT, undo with ROLLBACK, and roll back to a mid-point with SAVEPOINT. Then come **JOINS**, which combine rows from two tables using a related column: INNER gives only matches, LEFT keeps everything from the left table, RIGHT keeps everything from the right, FULL keeps everything, CROSS pairs every row with every row, and SELF joins a table to itself.

Example:

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
ROLLBACK TO after_debit;      -- undo only the credit
COMMIT;

-- JOIN example
SELECT c.name, o.amount
FROM customers c
INNER JOIN orders o ON c.id = o.customer_id;
```

Bank transfers are the classic transaction example: debit one account, credit the other, or do nothing.

Why it matters: JOINS and transactions are the two SQL topics interviewers ask about the most.

In the big merged file, this starts on page 123.

Part 4 - Web Scraping and Visualisation

PDFs 18 to 21 - collecting data from the web and showing it

PDF 18 Web Scraping

What is this? Web scraping means collecting information from websites automatically. To do it you first need to understand **HTML**, the language every web page is written in. HTML uses tags such as h1 for headings, p for paragraphs, a for links, img for images and table for tables, and tags carry attributes like id, class, href and src. Once you know that structure, you fetch the page with the **requests** library and pull out the pieces you want with **BeautifulSoup**, usually by tag name, class or id.

Example:

```
import requests
from bs4 import BeautifulSoup

page = requests.get("https://example.com")
soup = BeautifulSoup(page.text, "html.parser")

print(soup.title.text)          # page title

for link in soup.find_all("a"): # every link
    print(link.get("href"))

items = soup.select(".product-name") # by CSS class
print([i.text for i in items])
```

Always check a site's terms before scraping, and never hammer it with requests.

Why it matters: Scraping lets you build your own dataset when none exists - a genuinely useful superpower.

In the big merged file, this starts on page 145.

PDF 19 Data Visualisation - Matplotlib

What is this? Matplotlib turns numbers into pictures. Humans understand a chart far faster than a table, so charts are how you find patterns, spot odd values and explain results to others. This PDF walks through the basic plot structure and the main chart types: line plots for trends over time, bar charts for comparing categories, histograms for seeing how values are spread, scatter plots for relationships between two variables, and pie charts for parts of a whole. It also covers colours, line styles, markers, legends, subplots and saving the chart to a file.

Example:

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr"]
sales = [120, 150, 90, 200]

plt.plot(months, sales, marker="o", color="blue")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.savefig("sales.png")
plt.show()

plt.bar(months, sales) # bar chart
plt.hist(sales, bins=5) # histogram
plt.scatter(months, sales) # scatter
```

Why it matters: One good chart can replace a thousand rows of numbers in a meeting.

In the big merged file, this starts on page 152.

PDF 20 Data Visualisation - Seaborn

What is this? Seaborn sits on top of Matplotlib and makes statistical charts prettier with far less code. It works directly with Pandas DataFrames, so you just name the columns. Its plots fall into three families: relational (line, scatter), categorical (bar, box, violin, count) and distribution (histogram, density, pair plot). The **hue** parameter is the star - it colours everything by a category, so one chart can compare three things at once.

Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

sns.scatterplot(data=tips, x="total_bill", y="tip", hue="day")
plt.title("Bill vs Tip by Day")
plt.show()

sns.boxplot(data=tips, x="day", y="total_bill", hue="sex")
sns.heatmap(tips.corr(numeric_only=True), annot=True)
```

Why it matters: Seaborn is the fastest way to make your analysis look professional during exploration.

In the big merged file, this starts on page 167.

PDF 21 Data Visualisation - Assignment

What is this? Five charting tasks: draw a histogram of 200 random exam scores and mark the average line, scatter flipper length against body mass in the penguins dataset coloured by species, box-plot restaurant bills by day, scatter bills against tips split by sex and smoking, and build a pivot table of average tips by day and time shown as an annotated heatmap.

Example:

```
pivot = tips.pivot_table(values="tip",
                          index="day",
                          columns="time",
                          aggfunc="mean")
sns.heatmap(pivot, annot=True, fmt=".2f")
plt.title("Average tip by day and time")
```

Why it matters: These five charts are the standard toolkit of a data analyst's first report.

In the big merged file, this starts on page 171.

Part 5 - Machine Learning (Supervised)

PDFs 22 to 35 - teaching a computer to predict

What is Machine Learning? Read this first.

Normal programming is: you write the rules, the computer follows them and gives you the answer. Machine learning flips it: you give the computer the data **and** the answers, and it works out the rules by itself. Then you hand it new data and it predicts the answer.

Example: instead of writing rules to recognise a cat in a photo - which is nearly impossible - you show the computer 10,000 photos labelled 'cat' and 10,000 labelled 'dog'. It finds the patterns itself. That is machine learning.

There are two main kinds in this part of the course. **Regression** predicts a number, like a house price. **Classification** predicts a category, like 'spam' or 'not spam'.

PDF 22 Linear Regression

What is this? The simplest and most important ML algorithm. Imagine dots scattered on a graph - hours studied on the x-axis, marks scored on the y-axis. Linear regression draws the single straight line that passes closest to all the dots. That line is your model, written as $y = mx + c$. To find the best line, the algorithm measures how wrong it is using a **cost function** (usually Mean Squared Error), then slowly improves using **gradient descent** - imagine walking downhill blindfolded until you reach the bottom of the valley.

Example:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X = [[1], [2], [3], [4], [5]]      # hours studied
y = [30, 45, 50, 65, 80]          # marks

model = LinearRegression()
model.fit(X, y)                    # find the best line

print(model.predict([[6]]))         # predict for 6 hours
print("slope:", model.coef_, "intercept:", model.intercept_)
```

With several inputs (size, rooms, location) the same idea is called multiple linear regression.

Why it matters: It is the foundation of forecasting: prices, sales, demand, temperature.

In the big merged file, this starts on page 173.

PDF 23 Evaluation Metrics for Regression

What is this? How do you know if your prediction model is any good? These four numbers tell you. **MAE** is the average size of your mistakes, in the original units. **MSE** squares the mistakes first, so big errors get punished hard. **RMSE** takes the square root of MSE, bringing it back to normal units - it is the most reported one. **R-squared** goes from 0 to 1 and says how much of the variation your model explains; 0.85 means 85 percent.

Example:


```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("MAE :", mean_absolute_error(y_test, pred))
print("MSE :", mean_squared_error(y_test, pred))
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R2  :", r2_score(y_test, pred))

```

Lower is better for MAE, MSE and RMSE. Higher is better for R-squared.

Why it matters: A model without an evaluation score is just a guess. Metrics are how you prove it works.

In the big merged file, this starts on page 178.

PDF 24 Supervised ML Assignment 1 - House Price Prediction

What is this? Your first full ML project. A real estate company wants to predict the selling price of a house from its features - size, condition, year built, basement area, zone and more. You load the data, clean it, train a linear regression model, and score it. This is the classic 'House Prices' problem that every ML beginner solves.

Example:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("train.csv")
X = df[["LotArea", "OverallCond", "YearBuilt", "TotalBsmtSF"]].fillna(0)
y = df["SalePrice"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression().fit(X_train, y_train)
print(model.score(X_test, y_test))

```

Why it matters: This is the project to put first on your resume - it covers the complete ML workflow end to end.

In the big merged file, this starts on page 181.

PDF 25 Logistic Regression

What is this? Do not let the name fool you - logistic regression is used for **classification**, not for predicting numbers. It answers questions like 'will this customer leave, yes or no?'. It does this with the **sigmoid** function, an S-shaped curve that squeezes any number into a value between 0 and 1, which we read as a probability. Above 0.5 we say yes, below we say no. It cannot use MSE as its cost function because that gives a bumpy curve, so it uses **log loss** instead, which heavily punishes confident wrong answers.

Example:

```

from sklearn.linear_model import LogisticRegression

X = [[1], [2], [3], [4], [5]]      # hours studied
y = [0, 0, 0, 1, 1]                # 0 = fail, 1 = pass

model = LogisticRegression()
model.fit(X, y)

print(model.predict_proba([[3.5]])) # chance of each class
print(model.predict([[3.5]]))       # final answer: 0 or 1

```

predict_proba gives you the probability; predict gives you the final class.

Why it matters: Spam filters, loan approvals and disease screening all started with this simple idea.

In the big merged file, this starts on page 183.

What is this? A company is losing employees and wants to know who might resign next. You build a logistic regression model on 15 features - job satisfaction, salary, work-life balance, training hours, bonus, distance from home and so on - then improve it with **regularization**. Regularization adds a small penalty for making the model too complex, which stops it memorising the training data. L1 can zero out useless features; L2 keeps all weights small. You compare both and pick the winner.

Example:

```
from sklearn.linear_model import LogisticRegression

base = LogisticRegression(max_iter=1000)
l1 = LogisticRegression(penalty="l1", solver="liblinear", max_iter=1000)
l2 = LogisticRegression(penalty="l2", max_iter=1000)

for name, m in [("base", base), ("L1", l1), ("L2", l2)]:
    m.fit(X_train, y_train)
    print(name, m.score(X_test, y_test))
```

Why it matters: Regularization is the standard fix when a model does great on training data but poorly on new data.

In the big merged file, this starts on page 188.

What is this? Accuracy alone can lie. If only 1 in 100 emails is spam, a model that says 'not spam' every time is 99 percent accurate and completely useless. So we use the **confusion matrix**, a table of four boxes: true positive, true negative, false positive, false negative. From it we compute **precision** (of the ones I flagged, how many were right), **recall** (of the real ones, how many did I catch) and **F1**, the balance of the two. When missing a case is dangerous, such as in cancer detection, recall matters more. When false alarms are costly, precision matters more.

Example:

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))

# by hand:
# precision = TP / (TP + FP)
# recall    = TP / (TP + FN)
# F1       = 2 * precision * recall / (precision + recall)
```

Why it matters: Choosing the right metric is a business decision, not just a maths one.

In the big merged file, this starts on page 190.

What is this? The most intuitive algorithm in the course, based on the saying 'tell me who your friends are and I will tell you who you are'. To classify a new point, KNN finds the K closest points in the training data and takes a majority vote. Distance is usually measured with Euclidean distance - straight-line distance. KNN is called a **lazy learner** because it does not build a model during training; it simply stores the data and does the work when a question arrives. One warning: because it is based on distance, you must scale your features first, otherwise columns with big numbers dominate.

Example:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s = sc.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train_s, y_train)
print(model.predict(X_test_s))
```

Always choose an odd K (3, 5, 7) so there is no tie in the vote.

Why it matters: Great baseline model, and surprisingly hard to beat on small clean datasets.

In the big merged file, this starts on page 193.

PDF 29

Supervised ML Assignment 3 - Iris Classification

What is this? Botanists measure flowers by hand, which is slow. You build a system that predicts the species of an iris flower from four measurements. The task is to train three different models - KNN, logistic regression and Naive Bayes - compare them, and pick the best. An important note in this assignment: Iris is small, balanced and clean, so all three models will score very high. Real data is messy, so never expect these numbers in the real world.

Example:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

for name, m in [("KNN", KNeighborsClassifier(5)),
                ("LogReg", LogisticRegression(max_iter=200)),
                ("NaiveBayes", GaussianNB())]:
    m.fit(X_train, y_train)
    print(name, round(m.score(X_test, y_test), 3))
```

Why it matters: Comparing models on the same data is the daily routine of a machine learning engineer.

In the big merged file, this starts on page 195.

PDF 30

Scratch Implementation Assignment - KNN from Scratch

What is this? Build a KNN regressor without using scikit-learn. This is the exercise that proves you actually understand the algorithm rather than just calling a library. You compute distances by hand, sort them, pick the K nearest, and average their values to make a prediction.

Example:

```
import numpy as np

X_train = np.array([[1], [2], [3], [4], [5]])
y_train = np.array([2, 4, 6, 8, 10])

def knn_predict(x, k=3):
    d = np.abs(X_train - x).ravel()      # distances
    idx = np.argsort(d)[:k]              # k nearest
    return y_train[idx].mean()           # average

print(knn_predict(1.5))    # about 3.0
print(knn_predict(6))      # about 9.3
```

Why it matters: Interviewers love asking people to write KNN from scratch. This assignment prepares you for that.

In the big merged file, this starts on page 197.

PDF 31 CreditWise Loan System (Project)

What is this? A full case study. A bank approves loans by hand, which is slow, inconsistent and biased - good customers get rejected and risky ones get approved. You are hired to build a model that predicts approve or reject from income, employment status, age, marital status, dependents, credit score and loan details. Before modelling you must explore the data, handle missing values, encode text columns as numbers and scale the features.

Example:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("loans.csv")
df = pd.get_dummies(df, drop_first=True)  # text -> numbers
df = df.fillna(df.median(numeric_only=True))

X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)
print(LogisticRegression(max_iter=1000).fit(X_tr, y_tr).score(X_te, y_te))
```

Why it matters: This is how a real ML project brief reads - business problem first, model second.

In the big merged file, this starts on page 198.

PDF 32 Supervised ML Assignment 4 - ShopSmart Purchase Prediction

What is this? An e-commerce company recorded 12,330 browsing sessions and wants to know which visitors are likely to buy. You build a **Decision Tree** - a model that asks a series of yes-or-no questions about the data, like a game of 20 questions, until it reaches an answer. The catch: most sessions did not end in a purchase, so the data is **imbalanced** and accuracy would be misleading. You must judge the model with the **F1 score** and try to beat 0.55. **Pruning** - limiting the tree's depth - helps it generalise instead of memorising.

Example:

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score

tree = DecisionTreeClassifier(max_depth=5, min_samples_leaf=20)
tree.fit(X_train, y_train)

pred = tree.predict(X_test)
print("F1:", f1_score(y_test, pred))

```

max_depth and min_samples_leaf are the pruning knobs. Try several values and compare F1.

Why it matters: Imbalanced data is the norm in the real world: fraud, churn, disease, defects.

In the big merged file, this starts on page 200.

PDF 33 SVM Kernels

What is this? A Support Vector Machine draws the widest possible gap, or margin, between two classes. Data is rarely separable by a straight line, so SVMs use a **kernel** - a trick that lifts the data into a higher dimension where a straight line does work. Four kernels are covered: **Linear** is fast and safe, **Polynomial** captures feature interactions but can overfit, **RBF** is the flexible default and works well on most problems but is sensitive to its settings, and **Sigmoid** behaves like a small neural network but is usually weaker than RBF.

Example:

```

from sklearn.svm import SVC

for k in ["linear", "poly", "rbf", "sigmoid"]:
    m = SVC(kernel=k).fit(X_train, y_train)
    print(k, round(m.score(X_test, y_test), 3))

```

Start with linear for lots of features; try RBF when the boundary is clearly curved.

Why it matters: SVMs were the top choice for medium-sized datasets before deep learning took over.

In the big merged file, this starts on page 202.

PDF 34 Supervised ML Assignment 5 - Health Risk Classification

What is this? A research institute has health records for 9,800 people and wants to split them into 'healthy' and 'at higher risk' so it can choose the right participants for clinical trials. The data mixes numbers such as age and BMI with categories such as lifestyle factors. You build a classifier, evaluate it properly, and check that it treats both classes fairly rather than just predicting the majority one.

Example:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
print(classification_report(y_test, rf.predict(X_test)))

```

Why it matters: Healthcare problems show why recall matters: missing an at-risk person is the costly error.

In the big merged file, this starts on page 203.

PDF 35 SmartCart Clustering System

What is this? A change of direction: this is **unsupervised** learning, meaning the data has no answer column. Nobody tells the computer what is correct; it must find groups on its own. **Clustering** does exactly that - it puts

similar customers together. SmartCart has 2,240 customer records and wants segments such as big spenders, bargain hunters and inactive users, so it can send each group the right offer. K-Means is the standard algorithm: you pick how many groups you want, it finds their centres and assigns every customer to the nearest one.

Example:

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

Xs = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=4, random_state=42)
df["segment"] = km.fit_predict(Xs)

print(df.groupby("segment")[["Income", "Spent"]].mean())
```

Scale the data before clustering, otherwise the biggest-numbered column decides everything.

Why it matters: Customer segmentation drives real marketing decisions worth real money.

In the big merged file, this starts on page 205.

Part 6 - Git and GitHub

PDFs 36 to 37 - saving your work and showing it to the world

PDF 36 Git Cheat Sheet

What is this? Git is a version control system. Think of it as an unlimited undo button plus a time machine for your code: every time you reach a point worth keeping, you take a snapshot called a **commit**. You can jump back to any snapshot, see what changed, and work on new features in separate **branches** without breaking the main code. GitHub is the website where you store those snapshots online, so they are backed up and shareable.

Example:

```
git config --global user.name "Your Name"
git config --global user.email "you@mail.com"

git init                # start tracking this folder
git status              # what changed?
git add .               # choose files for the snapshot
git commit -m "first commit" # take the snapshot

git branch feature-x    # make a branch
git checkout feature-x  # switch to it
git merge feature-x     # bring it back into main

git clone <url>          # download a project
git push origin main    # upload your work
git pull origin main    # download updates
```

Why it matters: Every company expects you to know Git. Your GitHub profile is your real portfolio.

In the big merged file, this starts on page 208.

PDF 37 Git and GitHub - Task

What is this? A short but important assignment: push your completed projects - the CreditWise loan system and the SmartCart segmentation system - into separate GitHub repositories using the command line. Two rules are stressed. First, do not upload your dataset; either leave it out or list it in a file called `.gitignore`. Second, add a README describing what the project does, because a repository without a README looks unfinished.

Example:

```
# a simple .gitignore
data/
*.csv
__pycache__/

# then push
git add .
git commit -m "Add loan prediction project"
git remote add origin <your-repo-url>
git push -u origin main
```

Why it matters: A clean GitHub profile with two or three documented projects gets you interviews.

In the big merged file, this starts on page 210.

Part 7 - Deep Learning

PDFs 38 to 42 - neural networks for images and text

What is Deep Learning? Read this first.

A normal machine learning model needs you to decide which features matter - for example, telling it that petal length matters for flowers. A **neural network** is built from layers of small units inspired by the brain, and it learns the features by itself. Stack many layers and you get **deep learning**. It needs a lot of data and a lot of computing power, but on images, text, audio and video it beats everything else.

PDF 38 Popular CNN Architectures

What is this? A CNN (Convolutional Neural Network) is the type of neural network built for images. Instead of looking at the whole picture at once, it scans small patches for simple shapes - edges, corners, textures - and then combines them into bigger ideas like eyes, wheels or faces. This PDF walks through the famous designs: **LeNet**, the early digit recogniser; **AlexNet**, which started the deep learning boom in 2012 using ReLU and GPUs; **VGG**, simple but heavy; **GoogLeNet / Inception**, which runs several filter sizes in parallel; **ResNet**, which uses skip connections so networks can be hundreds of layers deep; and **MobileNet**, built to run fast on phones. The practical lesson is **transfer learning**: take a model already trained on millions of images and fine-tune it for your own task.

Example:

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

base = ResNet50(weights="imagenet", include_top=False,
                input_shape=(224, 224, 3))
base.trainable = False # freeze what it already knows

model = Sequential([base, GlobalAveragePooling2D(),
                    Dense(1, activation="sigmoid")])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
```

With transfer learning you can reach high accuracy with a few hundred images instead of millions.

Why it matters: Nobody trains a CNN from scratch any more unless they have to. Everyone fine-tunes a pre-trained model.

In the big merged file, this starts on page 211.

PDF 39 Binary Image Classification - Cats vs Dogs

What is this? Your first computer vision project: teach a CNN to tell a cat photo from a dog photo. The workflow is always the same - collect and resize images, normalise the pixel values, split into training and validation sets, build a stack of convolution and pooling layers ending in a dense output, train it, then check accuracy, precision, recall and the confusion matrix. A decent model here reaches 80 to 95 percent.

Example:


```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation="relu", input_shape=(150,150,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation="relu"),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation="relu"),
    Dense(1, activation="sigmoid") # 0 = cat, 1 = dog
])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])

```

Why it matters: The same pattern you learn here classifies X-rays, satellite images and factory defects.

In the big merged file, this starts on page 215.

PDF 40 Word2Vec

What is this? Computers cannot read words, they only understand numbers. The old method, one-hot encoding, gives each word a huge vector of zeros with a single one - and it captures no meaning at all, so 'cat' and 'dog' look as unrelated as 'cat' and 'table'. **Word2Vec** fixes this by learning a small, dense vector for every word from the company it keeps. Words used in similar situations end up close together in space, and the geometry even captures relationships.

Example:

```

# the famous example
# vector("king") - vector("man") + vector("woman")
# is closest to vector("queen")

from gensim.models import Word2Vec

sentences = [
    ["i", "love", "python"],
    ["python", "is", "fun"],
    ["i", "love", "coding"]
]

model = Word2Vec(sentences, vector_size=100, window=5, min_count=1)
print(model.wv.most_similar("python"))

```

Word2Vec has two training styles: CBOW predicts a word from its neighbours, Skip-gram does the reverse.

Why it matters: Word embeddings were the breakthrough that made modern language AI possible.

In the big merged file, this starts on page 216.

PDF 41 LSTM Architecture

What is this? Normal neural networks have no memory - they treat every input as brand new. An **RNN** fixes that by passing information forward, which makes it good for sequences like sentences or time series. But plain RNNs forget things that happened long ago, a problem called the vanishing gradient. An **LSTM** (Long Short-Term Memory) solves it with a memory line called the **cell state** plus three learned gates: the **forget gate** decides what to throw away, the **input gate** decides what new information to store, and the **output gate** decides what to pass on. Think of it as a notebook with an eraser, a pen and a highlighter.

Example:

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding

model = Sequential([
    Embedding(input_dim=10000, output_dim=128),
    LSTM(64),                    # remembers the sequence
    Dense(1, activation="sigmoid") # e.g. positive / negative review
])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])

```

Why it matters: LSTMs powered translation, speech recognition and sentiment analysis for years before Transformers arrived.

In the big merged file, this starts on page 217.

PDF 42 Handwritten Digit Classification - CNN and RNN

What is this? The MNIST project, the 'hello world' of deep learning. You are given 28x28 grayscale images of handwritten digits from 0 to 9 and you must classify them. The twist: build **two** models and compare them. The CNN treats the image as a picture and learns spatial features. The RNN treats each row of pixels as a step in a sequence and processes it like a sentence. Comparing them teaches you when to use which architecture. Both should score above 95 percent.

Example:

```

from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import SimpleRNN, LSTM, Conv2D

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0      # pixels from 0-255 to 0-1

# as an image -> CNN: Conv2D(...) then MaxPooling2D(...)
# as a sequence-> RNN: SimpleRNN(128) over the 28 rows

```

Why it matters: Same data, two ways of looking at it, two different results - that is architectural thinking.

In the big merged file, this starts on page 219.

Part 8 - Reinforcement Learning

PDFs 43 to 45 - learning by trial and error

What is Reinforcement Learning? Read this first.

Machine learning so far was learning from examples that already had answers. Reinforcement learning is different: there are no answers. There is an **agent** sitting in an **environment**. It takes an **action**, the environment gives it a **reward** or a penalty, and the agent slowly learns which actions bring the most reward over time.

This is how you trained a pet. You did not show it a labelled dataset of good and bad behaviour; it tried things, got a treat or a scolding, and adjusted. That is reinforcement learning.

PDF 43 Reinforcement Learning - Introduction

What is this? The vocabulary and the big picture. The **agent** is the learner. The **environment** is the world it acts in, such as a game or a robot's room. A **state** is where things stand right now. An **action** is a move the agent can make. A **reward** is the score it gets back. A **policy** is its strategy, the rule that says which action to take in which state. This is formalised as a Markov Decision Process. The central dilemma is **exploration versus exploitation**: do you try something new in the hope it is better, or stick with what already works? Too much exploring wastes time; too much exploiting means you never find the best option.

Example:

```
# the classic grid world: S = start, E = exit, X = wall
#   S   .   .   .
#   .   X   .   .
#   .   .   .   E
#
# agent gets -1 for every step (hurry up)
# and +10 when it finally reaches E
# its job: find the shortest safe path by trial and error
```

Why it matters: This is the technology behind game-playing AI, robotics, ad placement and recommendation systems.

In the big merged file, this starts on page 220.

PDF 44 Reinforcement Learning - Part 2 (DP, Monte Carlo, TD)

What is this? Three ways to learn the value of being in a state - that is, how good it is to stand here. **Dynamic Programming** needs a complete model of the world and uses the Bellman equation; it is exact but too slow for big problems. **Monte Carlo** methods need no model at all: they play whole episodes to the end, then average what happened - simple, but they must wait for the episode to finish. **Temporal Difference learning** is the sweet spot and the most important of the three: it updates after every single step by blending what it already believed with what it just observed, which is close to how humans actually learn.

Example:

```
# Q-learning: one line of TD learning
# Q(s,a) = Q(s,a) + alpha * [ reward + gamma * max(Q(next_s)) - Q(s,a) ]

import numpy as np

Q = np.zeros((n_states, n_actions))
alpha, gamma = 0.1, 0.9

Q[state, action] = Q[state, action] + alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)
```

gamma is the patience knob: high gamma cares about the long-term reward, low gamma wants reward now.

Why it matters: Q-learning is the root of most modern reinforcement learning, including the next PDF.

In the big merged file, this starts on page 236.

PDF 45 Deep Q-Networks (DQN) - Training Flappy Bird

What is this? Q-learning keeps a table of values, which is fine for a small grid but impossible when there are millions of states or continuous inputs like pixels. The fix is to replace the table with a neural network that estimates the value - that is a **Deep Q-Network**, the algorithm that learned to play Atari games from raw screen pixels. Two clever tricks make it stable. **Experience replay** stores past experiences and samples them randomly, because consecutive game frames are too similar and would confuse the network. A **target network** is a frozen copy used to compute the goal, because otherwise the target moves every step and training never settles. The PDF ends by training this on Flappy Bird.

Example:

```
# simplified DQN training step
states, actions, rewards, next_states, dones = replay_buffer.sample(64)

target = rewards + gamma * target_net.predict(next_states).max(axis=1)
target[dones] = rewards[dones]

q_values = policy_net.predict(states)
q_values[range(64), actions] = target

policy_net.fit(states, q_values, verbose=0)
```

Why it matters: Watching an agent go from crashing immediately to flying forever is the most satisfying moment in ML.

In the big merged file, this starts on page 252.

Part 9 - Transformers, Attention and Large Language Models

PDFs 46 to 47 - the idea behind ChatGPT

PDF 46 Attention Is All You Need (the original research paper)

What is this? The 2017 paper that changed artificial intelligence. Before it, language models read text word by word with RNNs or LSTMs, which was slow and forgot long-range connections. This paper proposed the **Transformer**, which throws away recurrence completely and relies only on **attention**. Attention lets every word look directly at every other word and decide how much each one matters, so in 'the animal didn't cross the street because it was too tired', the model can link 'it' straight to 'animal'. Because nothing has to wait for the previous word, training runs in parallel and far faster. The paper's model hit a record 28.4 BLEU on English-to-German translation while costing a fraction of the training time.

Example:

```
# self-attention in one formula
# Attention(Q, K, V) = softmax(Q * K_transpose / sqrt(d)) * V
#
# Q = what am I looking for?
# K = what does each word offer?
# V = the actual meaning I will take

import torch.nn.functional as F

scores = Q @ K.transpose(-2, -1) / (d ** 0.5)
weights = F.softmax(scores, dim=-1)
output = weights @ V
```

The paper also introduced multi-head attention and positional encoding, since attention alone has no sense of word order.

Why it matters: GPT, BERT, Claude, Gemini and translation tools are all descendants of this single paper.

In the big merged file, this starts on page 266.

PDF 47 Transformer Slides (GenAI and LLMs)

What is this? The classroom version of the same story, plus the bigger picture. It starts with the AI family tree: AI contains machine learning, which contains deep learning, which contains generative AI. It then explains **Large Language Models** - models with billions of parameters trained on huge amounts of text - and the Transformer architecture that powers them: turn text into **embeddings** (numbers that carry meaning), add position information, then pass everything through an **encoder** that understands the input and a **decoder** that generates the output. A key contrast is drawn between **static** embeddings, where a word has one fixed vector, and **contextual** embeddings, where the same word gets different vectors depending on the sentence - so 'book' in 'read a book' and 'book a ticket' are represented differently.

Example:

```
from transformers import pipeline

gen = pipeline("text-generation", model="gpt2")
print(gen("Python is popular because", max_length=30))

# sentiment uses the encoder side
clf = pipeline("sentiment-analysis")
print(clf("This course is genuinely useful!"))
```

Why it matters: The encoder-decoder split explains why some models are good at understanding and others at writing.

In the big merged file, this starts on page 277.

Part 10 - RAG and CSS (Web Basics)

PDFs 48 to 57 - making LLMs accurate, and styling web pages

PDF 48 CSS Part 1 - Practice Questions

What is this? A switch from Python to web design. CSS is the language that makes web pages look good while HTML provides the structure. In this assignment you are given an HTML page containing the poem 'Ozymandias' and asked to style it: change background colours, align text, set fonts, colour headings, decorate links and buttons, and underline a single word using a span element.

Example:

```
<!-- HTML -->
<h1>Ozymandias</h1>
<p>My name is <span>Ozymandias</span>, King of Kings;</p>

/* your task: write the CSS to style it */
```

Why it matters: HTML is the skeleton of a page, CSS is the clothes. You need both.

In the big merged file, this starts on page 303.

PDF 49 CSS Part 1 - Solutions

What is this? The worked answers. The pattern to notice: you select an element, then list properties inside curly braces. You can put CSS inline with a style attribute, inside a style block, or - the professional way - in a separate .css file linked from the HTML. Keeping CSS in its own file means one change updates every page.

Example:

```
p {
  color: brown;
  text-align: center;
  line-height: 30px;
}

h1 {
  text-align: center;
  color: red;
  font-family: Georgia;
}

a {
  text-decoration: none;
  color: green;
}

button {
  background-color: white;
  color: blueviolet;
}

span { text-decoration: underline; }
```

Why it matters: Try the questions yourself before opening this file - that is where the learning happens.

In the big merged file, this starts on page 306.

What is this? Goes deeper into **selectors**, the part before the curly braces that says which elements to style. You will use an id selector for one unique element, the universal selector for everything on the page, grouped selectors for several headings at once, a class selector so you can avoid the plain element selector, and a descendant selector for buttons inside a div. Part B moves to **pseudo-classes** such as hover for mouse-over effects, nth-of-type for picking specific children, first-letter for drop caps, and checked for form inputs.

Example:

```
/* id selector      */ #mainTopic { color: blue; }
/* universal        */ *           { text-align: center; }
/* grouped          */ h1, h3, h5 { font-family: Georgia; }
/* class            */ .content { background-color: cornflowerblue;
                             color: white; }
/* descendant       */ div button { background-color: purple; }
/* pseudo-class     */ button:hover { background-color: yellow; }
/* nth child        */ p:nth-of-type(2n+1) { color: yellow; }
/* first letter     */ h1::first-letter { color: red; }
```

Why it matters: Good selectors keep your CSS short. Bad selectors make you repeat yourself.

In the big merged file, this starts on page 308.

What is this? The answers, plus one question that trips up almost everyone: put four selectors in order of priority. **Specificity** decides which rule wins when two rules target the same element. The order is: an id beats everything, then a class, then a plain element name. So #mainContent beats div .main, which beats .main, which beats h1. Inline styles beat all of them, and !important beats even that - but use !important rarely, because it makes CSS hard to fix later.

Example:

```
#mainTopic { color: blue; }
div button { background-color: purple; color: azure; }
button:hover { background-color: yellow; color: blue; }
p:nth-of-type(2n+1) { color: yellow; }
h1::first-letter { color: red; }
#yes:checked + label { color: darkgreen; }

/* specificity: #id > .class > element */
```

Why it matters: When your CSS 'does not work', a specificity clash is the cause nine times out of ten.

In the big merged file, this starts on page 311.

What is this? One of the most useful ideas in modern AI. A plain language model has two problems: it makes things up (called hallucination) and its knowledge is frozen at training time, so it does not know about yesterday. **RAG** fixes both by letting the model look things up before answering. It works in two stages. During **ingestion**, your documents are split into chunks, converted into embeddings, and stored in a vector database. During **retrieval**, the user's question is embedded too, the closest chunks are found by similarity search, and those chunks are added to the prompt so the model answers using your actual documents.

Example:


```
# the idea in four steps
# 1. chunks = split(my_pdf_text)
# 2. store = vector_db.add(embed(chunks))
# 3. found = store.search(embed(user_question), k=5)
# 4. answer = llm("Use this context: " + found + "\nQ: " + question)

from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

db = Chroma.from_documents(chunks, OpenAIEmbeddings())
docs = db.similarity_search("What is our leave policy?", k=4)
```

Why it matters: RAG is how companies put ChatGPT on their own data without retraining anything.

In the big merged file, this starts on page 314.

PDF 53 Block and Inline Elements

What is this? A short but important concept that explains most CSS layout confusion. **Block** elements start on a new line and take the full width available - headings, paragraphs and divs behave this way. **Inline** elements sit inside the current line and only take the width their content needs - spans, links and images behave this way. Block elements respect width and height; inline elements ignore them. Vertical margins work on block elements but not on inline ones. Knowing which type you are dealing with saves hours of frustration.

Example:

```
div, h1, p { display: block; } /* full width, new line */
span, a, img { display: inline; } /* sits in the line */

/* the useful middle ground */
.card { display: inline-block; /* in a line, but width works */
        width: 200px; height: 100px; }

/* modern layouts */
.row { display: flex; gap: 12px; }
```

Why it matters: Most 'why is my layout broken?' questions come down to block versus inline.

In the big merged file, this starts on page 324.

PDF 54 CSS Part 3 - Practice Questions

What is this? The box model, which is how CSS thinks about space. Every element is a box made of content, padding inside it, a border around that, and margin outside it. You will build a box with exact dimensions, a dotted border and four different margin values, turn square divs into circles with border-radius, place them side by side with display, and convert a bulleted list into a single line - with a question asking you to notice and explain why the bullets vanish.

Example:

```
/* the box model */
#box {
    height: 150px;
    width: 300px;
    background-color: grey;
    border: 5px dotted black;
    margin: 20px 1em 40px 2em; /* top right bottom left */
    padding: 1em; /* space inside the box */
}
```

Why it matters: The margin order is always top, right, bottom, left - clockwise from 12 o'clock.

In the big merged file, this starts on page 325.

PDF 55 CSS Part 3 - Solutions

What is this? The answers, including the explanation for the disappearing bullets. List items are block-level elements with their display property set to list-item, and that is what draws the bullet. Setting display to inline overrides the default, so the bullet disappears. The lesson generalises: many CSS surprises are just default browser styles being overridden.

Example:

```
.box {
  height: 100px;
  width: 100px;
  background-color: pink;
  border: 2px solid black;
  border-radius: 50%;      /* makes it a circle */
  display: inline-block;   /* sit side by side */
}

li { display: inline; }    /* bullets disappear */
```

Why it matters: border-radius: 50% is the one-line trick behind every round avatar on the web.

In the big merged file, this starts on page 326.

PDF 56 CSS Part 4 - Practice Questions

What is this? Movement and polish. You will add a **transition** so a box smoothly grows when you hover over it, use **transform** to move and rotate an element and to skew another, add a **box-shadow** for depth, set a background image that covers its container, and reduce transparency with opacity. These few properties are what make a page feel designed rather than plain.

Example:

```
/* Q1: grow smoothly on hover */
#box1:hover { width: 600px; }
/* your job: add the transition to #box1 */

/* Q2 hint: move then rotate */
/* transform: translate(200px, 200px) rotate(90deg); */

/* Q3 hint: skew along x */
/* transform: skewX(20deg); */
```

Why it matters: Transitions and transforms are the cheapest way to make a page feel alive.

In the big merged file, this starts on page 329.

PDF 57 CSS Part 4 - Solutions

What is this? The completed code. Note the transition shorthand: property, duration, timing curve, delay. Transforms can be chained in a single line and are applied right to left. Box-shadow takes horizontal offset, vertical offset, blur radius and colour. Opacity runs from 0 (invisible) to 1 (solid).

Example:

```
#box1 { width: 100px; height: 100px; background: green;
        transition: width 2s ease-in 0.5s; }
#box1:hover { width: 600px; }

#box2 { transform: translate(200px, 200px) rotate(90deg); }
#box3 { transform: skewX(20deg); }
#box4 { box-shadow: 2px 2px 5px green; }

#myPic { width: 200px; height: 200px;
         background-image: url("picture.jpg");
         background-size: cover;
         opacity: 50%; }
```

Why it matters: These five properties cover most of the visual polish you see on real websites.

In the big merged file, this starts on page 332.

Part 11 - Generative AI and Deployment

PDFs 58 to 59 - creating new data, and sharing your work

PDF 58 GAN - Generative Adversarial Networks

What is this? Most models classify; a GAN **creates**. It puts two neural networks in a contest. The **Generator** makes fake data, starting from random noise, and tries to fool its opponent. The **Discriminator** looks at real and fake samples and tries to spot the difference. As the discriminator improves, the generator must improve to keep fooling it, and after enough rounds the fakes become almost indistinguishable from the real thing. It is a counterfeiter versus a detective, both getting better together. The PDF covers the vanilla version, DCGAN which uses convolution layers for images, and the two loss functions. GANs are famously unstable to train and hard to evaluate, which is partly why diffusion models are now popular for images.

Example:

```
# the training loop in words
for step in range(steps):
    noise = random_normal(batch, 100)
    fake_images = generator(noise)

    d_loss = discriminator.train_on_batch(real_images, ones)
    d_loss += discriminator.train_on_batch(fake_images, zeros)

    # now train the generator to make D say "real"
    g_loss = gan.train_on_batch(noise, ones)
```

Why it matters: GANs gave us the Ghibli-style art trend, deepfakes, face aging apps and realistic synthetic training data.

In the big merged file, this starts on page 335.

PDF 59 Streamlit Cheat Sheet

What is this? The fastest way to turn your Python script into a website. Streamlit is a library where you write plain Python - no HTML, no CSS, no JavaScript - and get an interactive web app with sliders, buttons, file uploads and charts. You run it with one command in the terminal and it opens in your browser. This is how you share a machine learning model with someone who will never open a code editor.

Example:

```
# app.py
import streamlit as st
import pandas as pd

st.title("Sales Dashboard")

file = st.file_uploader("Upload a CSV")
if file:
    df = pd.read_csv(file)
    st.dataframe(df.head())

    city = st.selectbox("Pick a city", df["City"].unique())
    st.line_chart(df[df["City"] == city]["Sales"])

    if st.button("Show summary"):
        st.write(df.describe())
```

run it with: # streamlit run app.py

Why it matters: Streamlit turns a model sitting in a notebook into something a manager can actually click.

In the big merged file, this starts on page 344.

Your Study Roadmap

How to go through all of this without getting lost

You do not have to learn everything at once. Here is a sensible order, based on how the topics depend on each other.

When	PDFs	What to do
Week 1-2	PDFs 1 to 10	Python basics. Type every example yourself. Do all five assignments.
Week 3	PDFs 11 to 13	NumPy and Pandas. Practise on a CSV file you care about.
Week 4	PDFs 14 to 17	SQL. Install MySQL and actually run the queries.
Week 5	PDFs 18 to 21	Scraping and charts. Build one small dataset and visualise it.
Week 6-8	PDFs 22 to 35	Machine learning. Start with linear and logistic regression, then trees and clustering.
Week 9	PDFs 36 to 37	Git and GitHub. Push every project from now on.
Week 10-11	PDFs 38 to 42	Deep learning. Start with transfer learning, not from-scratch CNNs.
Week 12	PDFs 43 to 45	Reinforcement learning. Optional if you are short on time.
Week 13	PDFs 46 to 47	Transformers. Understand attention and you understand modern AI.
Week 14	PDFs 48 to 57	CSS. Needed if you want to build web apps.
Week 15	PDFs 58 to 59	GANs and Streamlit. Deploy one project so others can use it.

Three habits that will make or break you

- **Type the code, never copy it.** Your fingers learn patterns that your eyes do not.
- **Break things on purpose.** Change a number, remove a line, see what error appears. Reading errors is a skill, and it is half of programming.
- **Finish small projects.** One finished project teaches more than ten half-read PDFs. The assignments in PDFs 24, 31, 35 and 39 are perfect candidates.

Glossary

40 words, explained in plain English

Word	Meaning
Algorithm	A fixed set of steps for solving a problem, like a recipe.
API	A door that lets one program request something from another program.
Array	A grid of numbers, usually from NumPy, that maths can be done on all at once.
Attention	The mechanism that lets a model decide which words matter most to which other words.
Attribute	A piece of data stored inside an object, such as a student's name.
Bias	The intercept in a line, or more generally a model's unfair tilt towards one answer.
Boolean	A value that is either True or False.
Branch	An independent line of work in Git, so you can experiment safely.
Class	A blueprint that describes the data and behaviour of the objects made from it.
Clustering	Grouping similar items together when nobody has told you the groups in advance.
CNN	Convolutional Neural Network. A neural network designed to understand images.
Commit	A saved snapshot of your code in Git.
Cost function	A number that says how wrong your model's predictions are. Training lowers it.
DataFrame	A table of data in Pandas with rows, columns and labels.
Deep learning	Neural networks with many layers that learn features by themselves.
Embedding	Turning a word, image or document into a list of numbers that carries its meaning.
Epoch	One complete pass of the model over the whole training dataset.
Feature	One input column your model learns from, such as the size of a house.
Function	A named bundle of code you can run again and again by calling its name.
Gradient descent	The method of slowly stepping downhill to find the lowest error.
Hyperparameter	A setting you choose before training, such as the learning rate or the value of K.
Imbalanced data	When one answer is far more common than the other, making accuracy misleading.
Iteration / loop	Repeating the same block of code many times.
Label	The correct answer attached to a row of training data.
Library	Ready-made code someone else wrote that you can install and use.
List comprehension	A short one-line way to build a new list from an existing one.
Model	Whatever your program learned from the data - the thing that makes predictions.
Normalization / scaling	Squeezing all columns onto a similar range so none of them dominates.

Object	One real thing created from a class, such as one specific student.
Overfitting	Memorising the training data instead of learning the pattern, so new data fails.
Pipeline	The full sequence: load data, clean it, train, evaluate, deploy.
Recall / Precision	Recall is how many real cases you caught; precision is how many of your guesses were right.
Regularization	A penalty that keeps a model simple so it works on new data.
Repository (repo)	A folder whose history Git is tracking, usually also stored on GitHub.
Sigmoid	An S-shaped curve that turns any number into a probability between 0 and 1.
Supervised learning	Learning from data where the correct answers are already given.
Token	A chunk of text a language model reads - roughly a word or part of a word.
Training / Testing set	Training data teaches the model; testing data checks it on questions it has never seen.
Transformer	The architecture built entirely on attention that powers modern language AI.
Vector database	A store built to find the most similar embeddings very quickly, used by RAG.

Final note

Nobody understands all of this on the first read. The people who look like experts simply went through the same material more than once. Read a topic here, open the matching PDF in the big merged file, type the example, break it, fix it - then move on. That loop is the whole secret.